

COMPENG 4DK4

Computer Communication Networks

Lab 1

September 22nd

Xikai Xu – L02 – xu503 - 400440917

Zihao Zhou – L02 – zhouz247 - 400473040

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by [**Zihao Zhou, zhouz247, 400473040**]

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by [**Xikai Xu, xu503, 400440917**]

Experiment 1:

1. **Experimental setup:** This experiment used a single-server infinite queue with a fixed service time of 1s. The arrival rate varied between 0.5, 0.9, 0.95, 0.99, 1.0, and 1.1, while the simulation run length varied among 10^2 , 10^4 , and 10^6 customers. The queue size was set unlimited, while 5 random seeds were used using the linear congruential generator $X_{n+1} = (aX_n + c) \bmod m$.

2. Results and plots:

Mean Delay Averaged Across Different Seeds:

Arrival rate	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
0.50	1.493	1.501	1.500
0.90	5.623	5.411	5.517
0.95	6.585	10.437	10.492
0.99	18.658	53.849	51.765
1.00	15.307	185.019	2943.759
1.10	64.774	5049.612	500807.211

Mean Delay at Arrival Rate 0.50 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
966620013	1.463551	1.499477	1.500649
1954778018	1.620024	1.512965	1.500551
1373241395	1.391663	1.501715	1.499851
1616105456	1.536177	1.492162	1.500305
1353374057	1.452508	1.498405	1.499784

Mean Delay at Arrival Rate 0.90 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
926072814	3.451205	5.358953	5.503550
1462398095	9.134710	5.515292	5.559103
1337850652	3.806673	5.340057	5.502030
1616105456	4.280098	5.520667	5.507151
1905192186	7.443930	5.319038	5.512567

Mean Delay at Arrival Rate 0.95 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
694895275	5.482522	9.901335	10.569067
1279110408	6.351856	9.541565	10.532622
489376929	5.920016	9.818771	10.488405
1112465094	7.114188	9.915388	10.389096
1452891783	8.055443	13.009969	10.480362

Mean Delay at Arrival Rate 0.99 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
2105840052	25.938810	43.071703	51.132625
990843357	35.333404	50.080810	54.465546
717889874	12.478216	35.210137	52.543264
1705204771	10.617278	93.868712	50.486413
850597152	8.920315	47.014687	50.198243

Mean Delay at Arrival Rate 1.00 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
493874905	13.720639	174.678689	1781.269642
2017788062	21.329647	119.675519	4320.753445
1405458303	14.728818	358.788668	3796.306558
1250190156	10.028579	154.412515	2908.795298
694656917	16.728827	117.537525	1911.670309

Mean Delay at Arrival Rate 1.10 for Different Seeds:

Seed	Mean delay for customer num 10^2	Mean delay for customer num 10^4	Mean delay for customer num 10^6
1979484330	66.057334	5364.923221	499888.836918
195439771	80.398472	4947.955412	500229.423731
1916430392	88.341751	5059.380931	499281.215241
1106223121	50.273846	5066.227497	503977.150224
871531894	38.798868	4809.572104	500659.429180

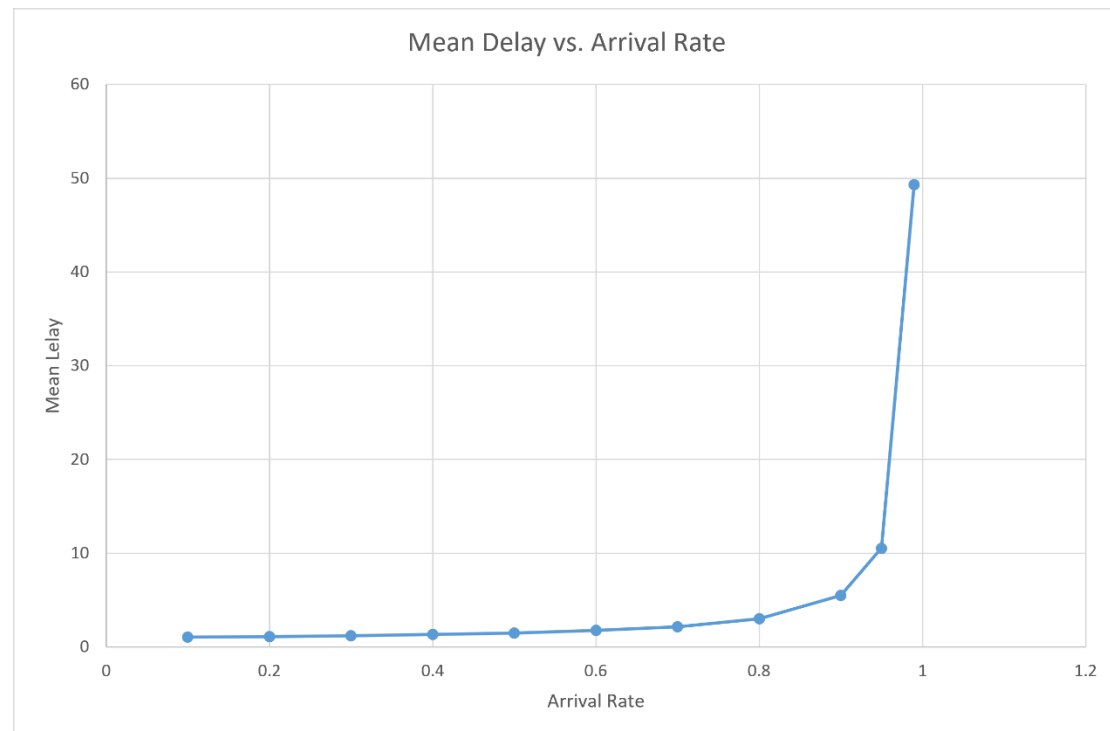
3. Discussion:

- i) The results are most sensitive to the random seed when the utilization is close to 1 and the number of customers served is small. For example, at $\rho = 0.99$, the mean delay varies much more between seeds than at $\rho = 0.5$. Similarly, at $\rho = 0.99$, the mean delay varies much more between seeds when only 10^2 customers are served than when 10^6 customers are served.
- ii) Run length becomes especially important when ρ is close to or greater than 1. For example, at $\rho = 1.0$, mean delay increases from 15.3 with 10^2 customers to 2943.8 with 10^6 customers. Clearly stable systems show much less sensitivity to run length.
- iii) Increase the simulation run length and repeat the experiment with different seeds. If mean delay, mean number in the system, and utilization converge to similar values, the results are likely representative of steady-state performance.
- iv) Yes, but only approximately. If the metrics converge as the simulation gets longer, the system is likely stable. If queue size and delay continue increasing with longer simulations, the system may be unstable. A finite simulation alone cannot prove stability.

4. **Analytical comparison:** For a stable M/D/1 queue with service time 1, the theoretical mean delay is $\bar{T} = \bar{n}/\lambda$ which is the Little's Theorem. Using the simulation results, the calculated mean delay from $\bar{n}/\lambda$ is very close to the simulated mean delay. The small differences are mainly due to random variation and the finite simulation run length. Overall, the results agree well with Little's Theorem.
5. **Simulation reliability:** Shorter runs and different random seeds cause more variation, especially when ρ is close to 1. Increasing the running length reduces this variation. Therefore, the 10^6 customer runs are preferred.
6. **Code modifications:** Added two for loop for different arrival rate and different seeds where seeds are calculated by $X_{n+1} = (aX_n + c) \bmod m$. Added the calculation for the average mean delay.

Experiment 2:

1. **Experimental setup:** This experiment used a single-server infinite queue with a fixed service time of 1s. The arrival rate varied between 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, and 0.99, while the simulation run length is fixed at 10^6 customers. The queue size was set unlimited, while 5 random seeds were used using the linear congruential generator $X_{n+1} = (aX_n + c) \bmod m$.
2. **Results and plots:**



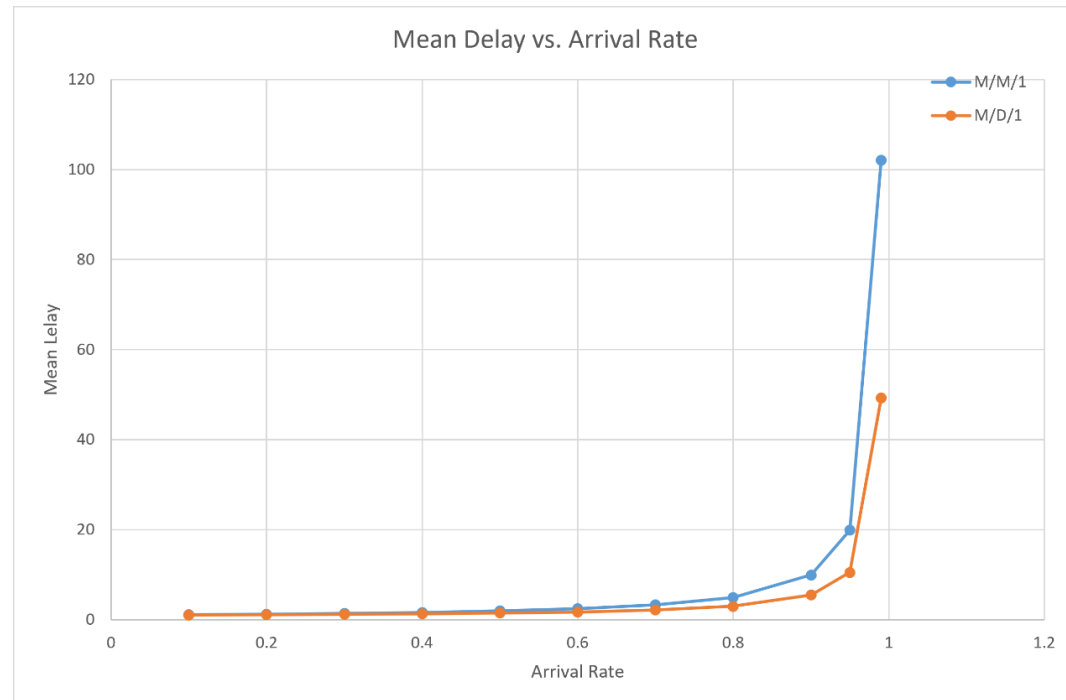
3. Discussion:

- i) At low arrival rates, the server is usually idle, so mean delay approaches the service time. Here, the number is 1.
- ii) As arrival rate increases, the server becomes busier, so the mean delay increases. As ρ approach to 1, delay approaches infinity. Therefore, the arrival rate asymptote is 1. The curve is nonlinear and becomes increasingly steep near 1 because there is very little spare server capacity, causing the queue to grow rapidly.
- iii) No. The code does not record individual customer delays or have a departure discipline, so it does not support collecting the variance of individual delays.

4. **Analytical comparison:** The simulation curve agrees with the theoretical prediction: mean delay increases as arrival rate increases and rises sharply as ρ approaches 1.
5. **Simulation reliability:** Longer simulation runs and different random seeds improve reliability. Seed effects are small at low ρ but become larger as ρ approaches 1. Therefore, the final curves use a long run of 10^6 customers and average results over multiple seeds to reduce random variation.
6. **Code modifications:** Change the arrival_rates array to 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, and 0.99.

Experiment 3:

1. **Experimental setup:** This experiment used a single-server infinite queue with an exponentially distributed service times with the mean of 1s. The arrival rate varied between 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, and 0.99, while the simulation run length is fixed at 10^6 customers. The queue size was set unlimited, while 5 random seeds were used using the linear congruential generator $X_{n+1} = (aX_n + c) \bmod m$.
2. **Results and plots:**



3. **Discussion:** Both M/D/1 and M/M/1 show the same trend: mean delay increases as arrival rate approaches 1. The M/M/1 mean delay is higher than M/D/1 because exponentially distributed service times have greater variance. This extra variability causes more queueing and therefore longer delays, especially as ρ approaches 1.
4. **Analytical comparison:** The simulation results closely match the theoretical predictions. The M/M/1 results follow the same trends as M/D/1, while M/D/1 has lower delay due to lower service-time variance.
5. **Simulation reliability:** Longer simulation runs and different random seeds improve reliability. Seed effects are small at low ρ but become larger as ρ approaches 1. Therefore, the final curves use a long run of 10^6 customers and average results over multiple seeds to reduce random variation.

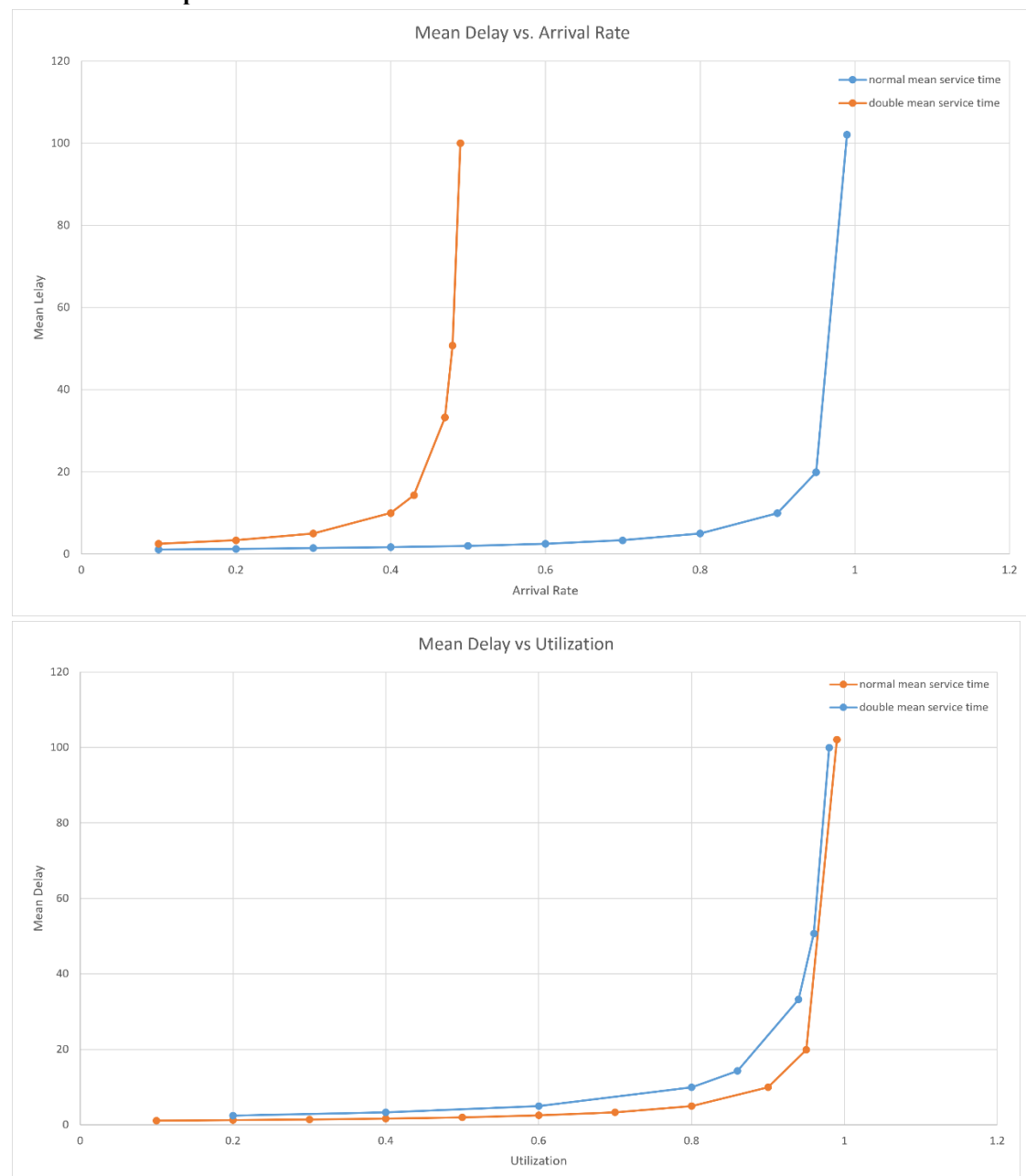
6. Code modifications: Change the service time to an exponential distribution.

```
if(number_in_system == 1) next_departure_time = clock
+exponential_generator((double)SERVICE_TIME);
total_busy_time += exponential_generator((double)SERVICE_TIME);
if(number_in_system > 0) next_departure_time = clock +
exponential_generator((double)SERVICE_TIME);
```

Experiment 4:

- 1. Experimental setup:** This experiment used a single-server infinite queue with an exponentially distributed service times with the mean of 2s. The arrival rate varied between 0.1, 0.2, 0.3, 0.4, 0.43, 0.47, 0.48, and 0.49, while the simulation run length is fixed at 10^6 customers. The queue size was set unlimited, while 5 random seeds were used using the linear congruential generator $X_{n+1} = (aX_n + c) \bmod m$.

2. Results and plots:



3. **Discussion:** Compared with Part 3, doubling the service time causes higher delay and more customers in the system for the same arrival rate as shown in the first plot. The system also becomes unstable at a lower arrival rate ($\lambda = 0.5$ instead of 1). Using ρ as the x-axis makes the curves easier to compare and shows that they have the same trends. They all show a sharp increase as ρ approaches 1, but the curve with a higher mean service time is less steep.
4. **Analytical comparison:** The simulation curve agrees with the theoretical prediction: mean delay increases as arrival rate increases and rises sharply as ρ approaches 1.
5. **Simulation reliability:** Longer simulation runs and different random seeds improve reliability. Seed effects are small at low ρ but become larger as ρ approaches 1. Therefore, the final curves use a long run of 10^6 customers and average results over multiple seeds to reduce random variation.
6. **Code modifications:** Change the arrival_rates array to 0.1, 0.2, 0.3, 0.4, 0.43, 0.47, 0.48, and 0.49 to avoid making the system overload. Also double the service time.

Experiment 5:

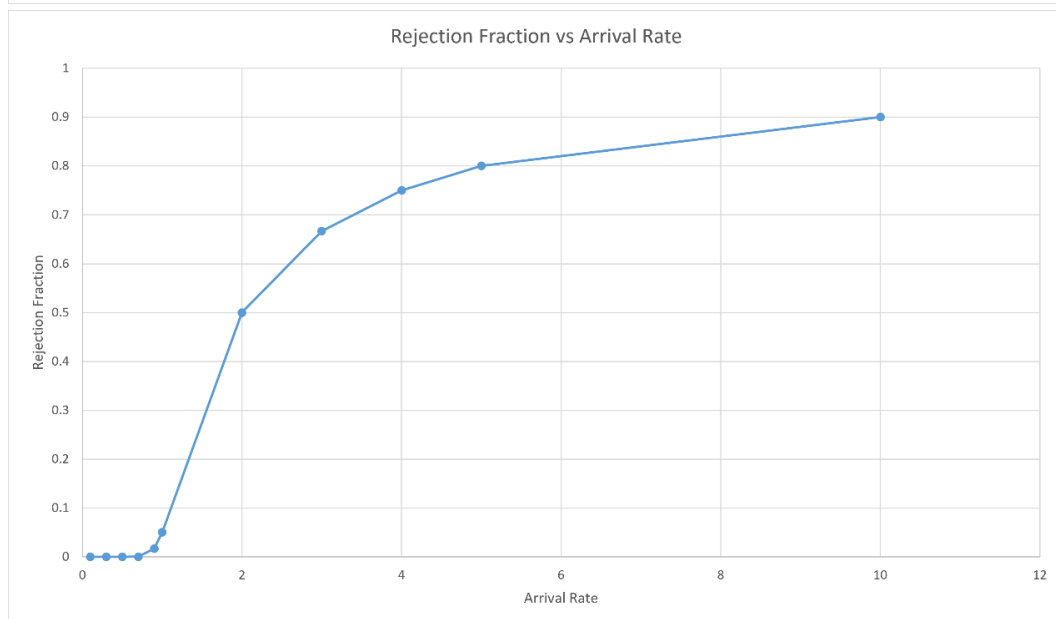
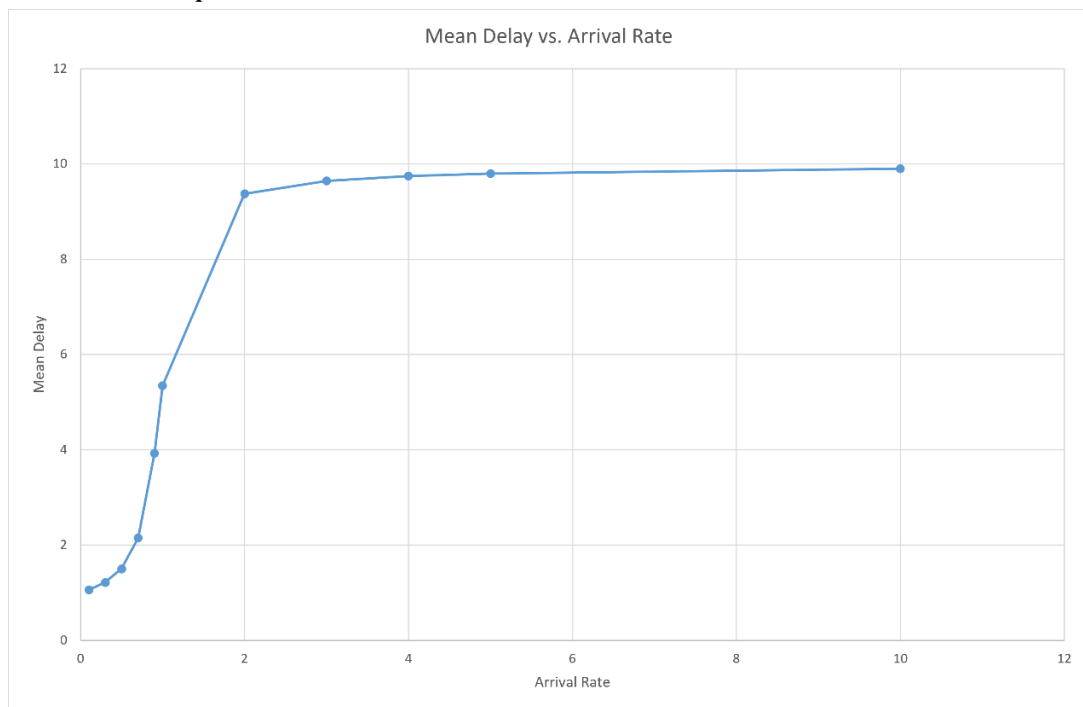
Arrival Rate	Mean Service Time	Utilization	System	Actual Mean Delay	Theoretical Mean Delay
0.1	1	0.1	M/D/1	1.055481	1.056
0.2	1	0.2	M/D/1	1.124963	1.125
0.3	1	0.3	M/D/1	1.214337	1.214
0.4	1	0.4	M/D/1	1.333346	1.333
0.5	1	0.5	M/D/1	1.500604	1.500
0.6	1	0.6	M/D/1	1.749234	1.750
0.7	1	0.7	M/D/1	2.165663	2.167
0.8	1	0.8	M/D/1	3.004435	3.000
0.9	1	0.9	M/D/1	5.513368	5.500
0.95	1	0.95	M/D/1	10.538629	10.500
0.99	1	0.99	M/D/1	49.291235	50.500
0.1	1	0.1	M/M/1	1.11078	1.111
0.2	1	0.2	M/M/1	1.24939	1.250
0.3	1	0.3	M/M/1	1.428342	1.429
0.4	1	0.4	M/M/1	1.666402	1.667
0.5	1	0.5	M/M/1	1.999497	2.000
0.6	1	0.6	M/M/1	2.499896	2.500
0.7	1	0.7	M/M/1	3.333427	3.333
0.8	1	0.8	M/M/1	4.992397	5.000
0.9	1	0.9	M/M/1	9.961771	10.000
0.95	1	0.95	M/M/1	19.914632	20.000
0.99	1	0.99	M/M/1	102.070307	100.000
0.1	2	0.2	M/M/1	2.498811	2.500
0.2	2	0.4	M/M/1	3.330277	3.333
0.3	2	0.6	M/M/1	5.002143	5.000
0.4	2	0.8	M/M/1	9.970692	10.000

0.43	2	0.86	M/M/1	14.259604	14.286
0.47	2	0.94	M/M/1	33.243643	33.333
0.48	2	0.96	M/M/1	50.68979	50.000
0.49	2	0.98	M/M/1	99.95638	100.000

The simulation results closely match the theoretical predictions. Small differences are due to random variation and finite simulation time.

Experiment 6:

- Experimental setup:** This experiment used a single-server infinite queue with a fixed service time of 1s. The arrival rate varied between 0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 2.0, 3.0, 4.0, 5.0, and 10.0, while the simulation run length is fixed at 10^6 customers. The queue size was fixed at 10, while 5 random seeds were used using the linear congruential generator $X_{n+1} = (aX_n + c) \bmod m$.
- Results and plots:**



3. Discussion:

- i) The maximum delay occurs when a customer arrives just after a departure, while the queue is still at its maximum size. The customer must wait for the maximum number of queued customers to be served, so the maximum delay is approximately the maximum queue size times the service time.
- ii) The maximum rejection fraction should close to 1 when the arrival rate is extremely high compared with the service rate, since almost every arriving customer is rejected when the queue is full and the service time is not short enough for new customers.

Compared with the infinite queue, the finite queue has bounded delay but can reject customers when the queue is full. The infinite queue does not reject customers, but the delay can grow without bound when the arrival rate exceeds the service rate.

It is most difficult to cover the full range with large MAX_QUEUE_SIZE, because higher arrival rates are required to fill the queue and produce a high rejection fraction.

- 4. **Analytical comparison:** The simulated maximum delay agrees with the theoretical maximum of the max queue size, while the rejection fraction approaches its theoretical maximum of 1 at very high arrival rates.
- 5. **Simulation reliability:** Longer simulation runs and different random seeds produce more stable results. The final curves use a sufficiently long run and multiple seeds to reduce random variation and improve reliability.
- 6. **Code modifications:** Change the arrival_rates array to 0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 2.0, 3.0, 4.0, 5.0, and 10.0.

Account for rejection cases:

```
7. total_arrived++;
8.
9.         if (number_in_system < MAX_QUEUE_SIZE) {
10.
11.             number_in_system++;
12.
13.             /* If this customer has arrived to an empty system, start
its service right away. */
14.             if(number_in_system == 1) next_departure_time = clock +
SERVICE_TIME;
15.
16.         } else {
17.
18.             total_rejected ++;
19.
20.         }
```

Calculate the rejection fraction:

```
total_rejection_fraction += (double) total_rejected/total_arrived;
average_rejection_fraction = total_rejection_fraction / 5;
```